

```
$ lab --phase 2 --num 1 --mode detailed
```

Lab 1 — Phase 2

Terraform — VPC from scratch

Procédure détaillée avec explication de chaque ligne de code

HCL

Variables

Resources

Count

Outputs

Outil

Auteur

Date

Statut

Terraform v1.15.7

Nathan

Juin 2026

Validé ✓

COURS

Concepts Terraform fondamentaux

1. Pourquoi Terraform ?

Terraform résout le problème de l'infrastructure manuelle : au lieu de cliquer dans la console AWS, on décrit l'infrastructure dans du code HCL. Ce code est versionné sur Git, reproductible, et déployable en une commande.

Reproductible	Le même code donne la même infrastructure à chaque fois. Fini les oublis de configuration.
Versionnable	L'historique Git montre tous les changements d'infrastructure. On peut revenir à un état précédent.
Collaboratif	Toute l'équipe travaille sur le même code, reviewé via Pull Request avant d'être appliqué.
Automatisable	Intégrable dans un pipeline CI/CD — l'infra se déploie automatiquement après validation.

2. Les 3 commandes essentielles

<code>terraform init</code>	Initialise le projet : télécharge les providers (plugins AWS, Azure...) dans .terraform/. À faire une seule fois par projet ou après ajout d'un provider.
<code>terraform plan</code>	Compare le code avec le state actuel et affiche ce qui va changer. Ne touche à rien dans AWS. C'est la commande de preview.
<code>terraform apply</code>	Applique les changements dans AWS. Demande une confirmation (yes) avant d'exécuter. Crée/modifie/supprime les ressources.
<code>terraform destroy</code>	Supprime TOUTES les ressources gérées par ce projet Terraform. Pratique pour nettoyer après un lab.

3. Le State — source de vérité

Le state (terraform.tfstate) est un fichier JSON qui enregistre toutes les ressources créées par Terraform avec leurs IDs AWS réels. C'est grâce à lui que Terraform sait ce qui existe déjà et ce qui doit changer.

■ State — règle critique

Le state ne doit JAMAIS être commité sur Git (il contient des informations sensibles). En équipe, on utilise un remote state stocké dans S3 avec un verrou DynamoDB. Sans ça, deux personnes qui font terraform apply en même temps peuvent corrompre l'infrastructure.

4. Structure HCL — les 4 blocs principaux

<code>terraform {}</code>	Configuration du projet : providers requis et leurs versions. Aussi utilisé pour configurer le backend (remote state).
---------------------------	--

<code>provider "aws" {}</code>	Configure le provider AWS : région, credentials. Terraform utilise ces infos pour parler à l'API AWS.
<code>resource "type" "nom" {}</code>	Déclare une ressource AWS à créer. Le type correspond au service AWS (aws_vpc, aws_subnet...). Le nom est un identifiant local.
<code>variable "nom" {}</code>	Paramètre configurable. Permet de rendre le code réutilisable sans le modifier.
<code>output "nom" {}</code>	Valeur affichée après apply. Utile pour récupérer des IDs générés par AWS (subnet-id, vpc-id...).

variables.tf

Explication ligne par ligne

```
variable "aws_region" {
  description = "Région AWS"
  type        = string
  default     = "eu-west-3"
}

variable "project_name" {
  description = "Nom du projet"
  type        = string
  default     = "phase2-lab1"
}

variable "vpc_cidr" {
  description = "Plage d'adresses IP du VPC"
  type        = string
  default     = "10.0.0.0/16"
}

variable "public_subnets" {
  description = "CIDRs des subnets publics"
  type        = list(string)
  default     = ["10.0.1.0/24", "10.0.2.0/24"]
}

variable "private_subnets" {
  description = "CIDRs des subnets privés"
  type        = list(string)
  default     = ["10.0.3.0/24", "10.0.4.0/24"]
}

variable "availability_zones" {
  description = "Zones de disponibilité"
  type        = list(string)
  default     = ["eu-west-3a", "eu-west-3b"]
}
```

description	Texte explicatif pour les humains — n'affecte pas le comportement.
type = string	La variable attend une chaîne de caractères. Autres types : number, bool, list(string), map(string).
type = list(string)	Une liste de chaînes. Accès par index : var.public_subnets[0] = '10.0.1.0/24'.
default = ...	Valeur utilisée si aucune valeur n'est fournie. Sans default, la variable est obligatoire.
var.nom_variable	Syntaxe pour utiliser une variable dans main.tf. Ex: var.vpc_cidr.

■ Principe DRY

Principe DRY (Don't Repeat Yourself) : on déclare une valeur UNE FOIS dans variables.tf et on l'utilise partout. Pour changer la région de Paris à Dublin, une seule ligne à modifier et tout le projet s'adapte.

main.tf

Explication bloc par bloc

Bloc 1 — Provider

```

terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

```

hcl

<code>required_providers</code>	Déclare les dépendances. Terraform télécharge ces plugins lors du terraform init.
<code>source = "hashicorp/aws"</code>	Adresse du provider sur le registry Terraform (registry.terraform.io).
<code>version = "~> 5.0"</code>	Accepte la version 5.x mais pas 6.0. Le ~> = compatible avec les minor versions.
<code>provider "aws"</code>	Configure le provider avec nos paramètres. Ici juste la région via variable.

Bloc 2 — VPC

```

resource "aws_vpc" "main" {
  cidr_block      = var.vpc_cidr
  enable_dns_hostnames = true
  enable_dns_support = true

  tags = {
    Name     = "vpc-${var.project_name}"
    Project = var.project_name
  }
}

```

hcl

<code>resource "aws_vpc" "main"</code>	Type = aws_vpc (ressource Terraform pour un VPC). Nom local = main (utilisé pour les références).
<code>cidr_block = var.vpc_cidr</code>	Utilise la variable — donne 10.0.0.0/16.
<code>enable_dns_hostnames</code>	Permet aux instances d'avoir un DNS automatique (ip-10-0-1-234.eu-west-3...).
<code>"vpc-\${var.project_name}"</code>	Interpolation de variable dans une chaîne. Donne vpc-phase2-lab1.
<code>aws_vpc.main.id</code>	Syntaxe pour référencer l'ID de ce VPC ailleurs dans le code.

Bloc 3 — Subnets avec count

```
resource "aws_subnet" "public" {  
  count          = length(var.public_subnets)  
  vpc_id         = aws_vpc.main.id  
  cidr_block     = var.public_subnets[count.index]  
  availability_zone = var.availability_zones[count.index]  
  
  map_public_ip_on_launch = true  
  
  tags = {  
    Name = "subnet-public-${count.index + 1}-${var.project_name}"  
  }  
}
```

<code>count = length(...)</code>	<code>length(var.public_subnets) = 2</code> . Terraform crée 2 ressources identiques. Évite la duplication de code.
<code>count.index</code>	Compteur automatique : 0 pour la première ressource, 1 pour la deuxième, etc.
<code>var.public_subnets[count.index]</code>	Accès par index : [0] = 10.0.1.0/24, [1] = 10.0.2.0/24.
<code>count.index + 1</code>	Dans le tag Name : +1 pour avoir 1 et 2 au lieu de 0 et 1 (plus lisible).
<code>aws_subnet.public[0].id</code>	Pour référencer un subnet spécifique. <code>[*].id</code> donne tous les IDs.

■ Count — scalabilité automatique

`count` est la façon la plus simple de créer plusieurs ressources similaires. Si demain tu ajoutes une 3ème AZ dans la variable, Terraform crée automatiquement un 3ème subnet sans toucher au `main.tf`.

Bloc 4 — Route table et associations

```

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main.id
  }

  tags = {
    Name = "rt-public-${var.project_name}"
  }
}

resource "aws_route_table_association" "public" {
  count          = length(var.public_subnets)
  subnet_id     = aws_subnet.public[count.index].id
  route_table_id = aws_route_table.public.id
}

```

<code>route {}</code>	Bloc imbriqué définissant une règle de routage. Équivaut à la ligne 0.0.0.0/0 → IGW qu'on ajoutait manuellement.
<code>gateway_id = aws_internet_gateway.main.id</code>	Référence l'IGW créée dans le même projet. Terraform détecte la dépendance et crée l'IGW d'abord.
<code>aws_route_table_association</code>	Associe un subnet à une route table. Avec count, boucle sur les 2 subnets publics.

outputs.tf — Valeurs exposées après apply

```

output "vpc_id" {
  description = "ID du VPC créé"
  value      = aws_vpc.main.id
}

output "public_subnet_ids" {
  description = "IDs des subnets publics"
  value      = aws_subnet.public[*].id
}

```

<code>output "vpc_id"</code>	Affiche l'ID du VPC après apply. Utile pour le réutiliser dans un autre module Terraform.
<code>[*].id</code>	Splat expression : récupère l'attribut id de TOUS les éléments du tableau. Donne une liste d'IDs.

PROCÉDURE

Ce qu'on a fait étape par étape

Étape 1 — Installation Terraform

```
bash
# Ajouter le repo HashiCorp
wget -O - https://apt.releases.hashicorp.com/gpg | \
  sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg

echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] \
  https://apt.releases.hashicorp.com $(lsb_release -cs) main" | \
  sudo tee /etc/apt/sources.list.d/hashicorp.list

sudo apt update && sudo apt install terraform -y

# Vérification
terraform version
# → Terraform v1.15.7
```

Étape 2 — Configurer l'accès AWS

Créer un utilisateur IAM dédié à Terraform dans la console AWS :

```
bash
IAM → Users → Create user
  Name      : terraform-user
  Policy    : AdministratorAccess (pour le lab)

IAM → terraform-user → Security credentials → Create access key
  Use case  : Command Line Interface (CLI)
  → Télécharger le CSV avec Access Key ID + Secret

# Sur le serveur Ubuntu :
aws configure
  AWS Access Key ID      : <ton access key>
  AWS Secret Access Key  : <ton secret key>
  Default region         : eu-west-3
  Default output format  : json

# Vérification
aws sts get-caller-identity
# → Affiche l'ARN de terraform-user
```

■ Sécurité — clés IAM

En production, on utilise des rôles IAM plutôt que des clés statiques. Les clés d'accès sont moins sécurisées car elles ne expirent pas automatiquement. Pour ce lab c'est acceptable — en prod jamais.

Étape 3 — Structure du projet

```
bash
mkdir -p ~/lab1-phase2-terraform-vpc
cd ~/lab1-phase2-terraform-vpc
touch main.tf variables.tf outputs.tf

# Structure du projet :
lab1-phase2-terraform-vpc/
■■■ main.tf          ← ressources AWS
■■■ variables.tf     ← paramètres configurables
■■■ outputs.tf       ← valeurs exposées après apply
■■■ .gitignore       ← exclut .terraform/ et *.tfstate
```

Étape 4 — Déploiement

```
bash
# Initialiser Terraform (télécharge le provider AWS)
terraform init

# Vérifier le plan sans rien créer
terraform plan
# → Plan: 12 to add, 0 to change, 0 to destroy.

# Créer l'infrastructure
terraform apply
# → Taper "yes" pour confirmer
# → Apply complete! Resources: 12 added, 0 changed, 0 destroyed.

# Outputs affichés :
# igw_id          = "igw-0544fa6e0c0e72cef"
# private_subnet_ids = ["subnet-...", "subnet-..."]
# public_subnet_ids  = ["subnet-...", "subnet-..."]
# vpc_id           = "vpc-007ba14b76c14488b"

# Supprimer l'infrastructure
terraform destroy
# → Destroy complete! Resources: 12 destroyed.
```

VALIDATION

Checklist finale

✓	Terraform installé	terraform version affiche v1.15.7
✓	AWS configuré	aws sts get-caller-identity affiche terraform-user
✓	variables.tf	6 variables déclarées (région, projet, CIDRs, AZs)
✓	main.tf	Provider + VPC + IGW + 4 subnets + 2 route tables + associations
✓	outputs.tf	vpc_id, public_subnet_ids, private_subnet_ids, igw_id
✓	terraform init	Provider AWS v5.100.0 installé sans erreur
✓	terraform plan	12 ressources à créer, 0 à modifier, 0 à détruire
✓	terraform apply	12 ressources créées, outputs affichés
✓	Vérification console	vpc-phase2-lab1 visible dans la console AWS
✓	terraform destroy	12 ressources supprimées proprement
✓	.gitignore	.terraform/ et *.tfstate exclus du repo

■ Prochaine étape — Remote State

Ce lab démontre la puissance de Terraform : 12 ressources AWS créées en 3 fichiers de code et 2 commandes. En Phase 1 la même infrastructure demandait plusieurs heures de clics. Prochaine étape : remote state S3 + DynamoDB pour travailler en équipe.